

ŽILINSKÁ UNIVERZITA V ŽILINE

Fakulta riadenia a informatiky

Semestrálna práca 1 Diskrétna simulácia

Michael Holý

Simulácia nákladov – Podnikateľ Ján

Cvičiaci : Ing. Peter Jankovič, Phd.

Žilina, 2025

Obsah

1	Zadanie semestrálnej práce	3
2	Popis jednotlivých stratégií.....	4
2.1	Vopred zadefinované stratégie	4
2.1.1	Výsledky vopred zadefinovaných stratégií	4
2.2	Navrhnutá vlastná stratégia	5
3	Popis implementácie generátorov pre jednotlivé rozdelenia	6
3.1	Trieda SeedGenerator	6
3.2	Trieda DiscreteUniformGenerator	6
3.3	Trieda ContinuousUniformGenerator	6
3.4	Triedy DiscreteEmpiricalTriple a ContinuousEmpiricalTriple.....	7
3.5	Trieda DiscreteEmpiricalGenerator	7
3.6	Trieda ContinuousEmpiricalGenerator	8
4	Popis implementácie simulácie Monte Carlo	9
4.1	Trieda GeneralMonteCarloSimulation	9
4.2	Trieda CustomOrder	9
4.3	Trieda MCSimBusinessmanJohn	10
4.3.1	Použité generátory	10
4.3.2	Inicializácia.....	10
4.3.3	Načítavanie konfigurácie z CSV súborov	11
4.3.4	Priebeh simulácie – beforeAllReplications().....	11
4.3.5	Priebeh simulácie – beforeReplication()	12
4.3.6	Priebeh simulácie – virtualReplication()	12
4.3.7	Priebeh simulácie – afterReplication()	13
4.4	Trieda BusinessmanJohnApplication	13
4.4.1	Inicializácia.....	13
4.4.2	Nastavenie vykresľovanie grafov	13
4.4.3	Spustenie simulácií	13
4.4.4	Zastavenie simulácie	13
5	Popis implementácie GUI.....	14
5.1	Trieda AppGUI	14
6	Záver	15

1 Zadanie semestrálnej práce

Cieľom tejto semestrálnej práce je optimalizovať stratégiu objednávania súčiastok v sklade podnikateľa Jána pomocou simulácie Monte Carlo. Ján nakupuje tlmiče, brzdové doštičky a svetlomety od dvoch dodávateľov, pričom musí minimalizovať celkové náklady na skladovanie a pokuty za nedodanie súčiastok počas 30-týždňového obdobia.

Úlohy:

- Analyzovať a porovnať štyri vopred definované stratégie objednávania.
- Navrhnuť vlastnú stratégiu vrátane plánovania objednávok na 30 týždňov.
- Implementovať flexibilný generátor spojitého a diskrétného rovnomerného a empirického rozdelenia.
- Implementovať simuláciu v programe s grafickým znázornením vývoja nákladov pre jednotlivé stratégie pri rastúcom počte opakovaní experimentu.
- Dodržať princípy oddelenia užívateľského rozhrania od jadra aplikácie.

2 Popis jednotlivých stratégií

Podnikateľ Ján objednáva určitý počet súčiastok, ktoré sú mu dodávané každý pondelok dodávateľom 1 alebo dodávateľom 2. Tieto súčiastky sú následne každý piatok v náhodnom množstve odkúpené odberateľom za fixné ceny, pričom pri nedodaní daného počtu súčiastok musí Ján zaplatiť pokutu.

Cieľom týchto stratégií je minimalizovanie týchto celkových nákladov – čiže náklady na skladovanie súčiastok a nákladov za pokuty.

2.1 Vopred zadané stratégie

Pre vopred zadané stratégie sú každý týždeň objednávky súčiastok pre dodávateľov pevne zadané na 100 ks tlmičov, 200 ks brzdových doštičiek a 150 ks svetlometov.

Jednotlivé stratégie:

- Stratégia A – Každý týždeň Ján objednáva súčiastky od dodávateľa 1.
- Stratégia B - Každý týždeň Ján objednáva súčiastky od dodávateľa 2.
- Stratégia C – Ján objednáva každý nepárny týždeň od dodávateľa 1 a každý párny od dodávateľa 2.
- Stratégia D - Ján objednáva každý nepárny týždeň od dodávateľa 2 a každý párny od dodávateľa 1.

2.1.1 Výsledky vopred zadaných stratégií

Počet replikácií	A	B	C	D
100 000	23078	11616	15957	15711
1 000 000	23097	11626	15958	15678
10 000 000	23085	11626	15950	15683

Na základe výsledkov zadaných stratégií sa javí **stratégia B ako najlepšia**, keďže priemerné celkové náklady pre daný počet replikácií sú vo všetkých prípadoch **najnižšie**. To je ale spôsobené tým, že náklady za skladovanie všetkých súčiastok sú vo väčšine prípadov

väčšie ako iba zaplataenie pokuty za ten daný týždeň. V prípade stratégie B odoberá Ján súčiastky iba od dodávateľa 2, ktorý je pri pohľade na rozdelenie pravdepodobností dodania súčiastok menej spoľahlivý ako dodávateľ 1 a tým pádom Ján častejšie platí iba pokutu za nedodanie súčiastky ako vynakladá náklady na jej skladovanie.

V prípade, že by sme brali ohľad aj na to, aby bol každý piatok odberateľovi dodaný čo najväčší počet súčiastok a tým pádom by bol uspokojený čo najviac, by sa z tohto pohľadu javila ako **dobrá** aj **stratégia A**, keďže dodávateľ 1 je spoľahlivejší ako dodávateľ 2.

2.2 Navrhnutá vlastná stratégia

V rámci semestrálnej práce som navrhol stratégiu, ktorá spočívala v objednávaní súčiastok na základe strednej hodnoty E pre dané rozdelenia, ktoré simulovali náhodný počet súčiastok požadovaných odberateľom. Stredná hodnota udáva priemerný očakávaný počet požadovaných súčiastok, a teda pri opakovaní viacerých pokusov by sa počet približoval práve k tejto hodnote. Preto som sa každý týždeň rozhodol objednávať počet súčiastok rovný E , čím som sa snažil optimalizovať veľkosť objednávky. Výsledky pre jednotlivé stredné hodnoty rozdelení E boli:

- $E(\text{Tlmiče}) = 75$
- $E(\text{Brzdové doštičky}) = 155$
- $E(\text{Svetlomety}) = 92$

Typ súčiastky	Typ rozdelenia	Množstvo
Tlmiče	diskrétné rovnomerné	$U_{\min} = 50, U_{\max} = 100$
Brzdové doštičky	diskrétné rovnomerné	$U_{\min} = 60, U_{\max} = 250$
Svetlomety	diskrétné empirické	$U = <30, 60>; p = 0.2$ $U = <60, 100>; p = 0.4$ $U = <100, 140>; p = 0.3$ $U = <140, 160>; p = 0.1$

Každý týždeň som ako dodávateľa zvolil dodávateľa 1, keďže bol spoľahlivejší spomedzi dvoch dostupných možností. Po vykonaní 10 000 000 replikácií simulácie sa priemerné celkové náklady ustálili na približne 9398. Touto navrhnutou stratégiou sa mi podarilo dosiahnuť **lepšie celkové náklady** ako v **predošlých stratégiách**, pričom aj odberateľ mal zrejme väčšiu spokojnosť, keďže dodávateľ 1 bol spoľahlivejší.

3 Popis implementácie generátorov pre jednotlivé rozdelenia

V tejto časti sa venujem popisu implementácie generátorov náhodných čísiel, ktoré som využíval v tejto semestrálnej práci.

3.1 Trieda SeedGenerator

Trieda **SeedGenerator** je zodpovedná za generovanie náhodných seedov pre generátory náhodných čísiel, ktoré využívame počas simulácií.

Ostatné generátory dostávajú iba referenciu na tú istú inštanciu tejto triedy, čo nám umožňuje v prípade potreby ladenia programu a aplikácie, zmeniť seed na jednom mieste (generátor seedov) a tým zabezpečiť, že všetky závislé generátory budú produkovať tie isté výsledky.

3.2 Trieda DiscreteUniformGenerator

Trieda **DiscreteUniformGenerator** predstavuje generátor diskrétného rovnomerného rozdelenia, ktorý generuje náhodné celé čísla v rozsahu $\langle uMin, uMax \rangle$.

Konštruktor prijíma dolnú ($uMin$) a hornú ($uMax$) hranicu intervalu, v ktorom sa majú generovať čísla, pričom generátor náhodných čísiel v tejto triede je inicializovaný pomocou seedu vygenerovaným generátorom seedov.

Metóda nextInt() vygeneruje a vráti náhodné celé číslo z intervalu $\langle uMin, uMax \rangle$.

3.3 Trieda ContinuousUniformGenerator

Trieda **ContinuousUniformGenerator** predstavuje generátor spojitého rovnomerného rozdelenia, ktorý generuje náhodné desatinné čísla v rozsahu $\langle uMin, uMax \rangle$.

Konštruktor prijíma dolnú ($uMin$) a hornú ($uMax$) hranicu intervalu, v ktorom sa majú generovať čísla, pričom generátor náhodných čísiel v tejto triede je inicializovaný pomocou seedu vygenerovaným generátorom seedov.

Metóda nextDouble() vygeneruje a vráti náhodné desatinné číslo v rozsahu $\langle uMin, uMax \rangle$.

3.4 Triedy **DiscreteEmpiricalTriple** a **ContinuousEmpiricalTriple**

Triedy **DiscreteEmpiricalTriple** a **ContinuousEmpiricalTriple** slúžia na uchovávanie intervalov pre generátory empirického rozdelenia.

Rozdiel medzi nimi je v podstate iba v type údajov, ktoré uchovávajú:

- **DiscreteEmpiricalTriple** – uchováva interval pre diskkrétne (celé) čísla.
- **ContinuousEmpiricalTriple** – uchováva interval pre spojité (desatinné) čísla.

Interval je definovaný v oboch triedach spodnou (uMin) a hornou (uMax) hranicou intervalu a taktiež aj pravdepodobnosťou (probability) s akou môže interval nastať.

3.5 Trieda **DiscreteEmpiricalGenerator**

Trieda **DiscreteEmpiricalGenerator** predstavuje generátor diskrétného empirického rozdelenia, v ktorom sú hodnoty generované podľa zadaných intervalov a ich pravdepodobností.

Konštruktor prijíma pole obsahujúce inštancie triedy **DiscreteEmpiricalTriple**, ktoré obsahuje jednotlivé intervaly a ich pravdepodobnosti.

Následne je pomocou generátora seedov inicializovaný generátor náhodných čísel pre výber intervalu a taktiež je aj pre každý interval inicializovaný vlastný generátor. Taktiež sa overuje, či súčet všetkých pravdepodobností v poli intervalov je rovný 1, inak sa vyvolá výnimka.

V **metóde nextInt()** sa najskôr náhodne vyberie interval na základe vygenerovanej pravdepodobnosti. Následne sa prechádza cez intervaly, pričom sa sčítavajú ich pravdepodobnosti, až kým sa nenájde zodpovedajúci interval pre vygenerovanú pravdepodobnosť. Zo zvoleného intervalu sa následne vygeneruje náhodné celé číslo v rozsahu <uMin, uMax) pomocou generátora pre ten daný interval.

3.6 Trieda ContinuousEmpiricalGenerator

Trieda **ContinuousEmpiricalGenerator** predstavuje generátor spojitého empirického rozdelenia, v ktorom sú hodnoty generované podľa zadaných intervalov a ich pravdepodobností.

Implementácia **Konštruktora** a metódy **nextDouble()** sú takmer identické implementácií v triede **DiscreteEmpiricalGenerator**, až na to, že v konštruktore sa prijíma pole obsahujúce inštancie triedy **ContinuousEmpiricalTriple** a v metóde **nextDouble()** sa po vybraní konkrétneho intervalu vygeneruje spojité (desatinné) číslo z rozsahu $[uMin; uMax)$ z daného intervalu.

4 Popis implementácie simulácie Monte Carlo

V tejto časti sa venujem popisu implementácie všeobecného jadra pre simulácie Monte Carlo, ako aj popisu implementácie simulácie pre podnikateľa Jána a aplikácie, ktorá vznikla pre riadenie týchto simulácií.

4.1 Trieda **GeneralMonteCarloSimulation**

Trieda **GeneralMonteCarloSimulation** predstavuje všeobecné simulačné jadro pre simulácie metódou Monte Carlo. Definuje štruktúru na spúšťanie opakovaných simulácií (replikácií), pričom umožňuje ich zastavenie pomocou metódy **stopSimulation()**.

Všetky podtriedy musia implementovať tieto metódy aby definovali svoju vlastnú simuláciu Monte Carlo:

- **beforeReplication()** – časť simulácie vykonaná pred samotnou replikáciou
- **virtualReplication()** – hlavná časť simulácie
- **afterReplication()** – časť simulácie vykonaná po replikácii (napr. na spracovanie výsledkov).

Taktiež táto trieda umožňuje podtriedam prekryť metódy **beforeAllReplications()** a **afterAllReplications()** v prípade, že potrebujú implementovať časti simulácie, ktoré sa vykonávajú pred a po všetkých vykonaných replikáciách.

4.2 Trieda **CustomOrder**

Trieda **CustomOrder** slúži na uchovávanie údajov o objednávke súčiastok, s ktorými sa pracuje neskôr v implementácii simulácie, ktorá bola zadefinovaná v zadaní tejto semestrálnej práce.

Obsahuje informácie o počte rôznych dielov a identifikátore dodávateľa, ktorý súčiastky dodáva.

4.3 Trieda MCSimBusinessmanJohn

Trieda **MCSimBusinessmanJohn** rozširuje všeobecné jadro **GeneralMonteCarloSimulation** a využíva **náhodné generátory** na simuláciu priebehu objednávaní, skladovania a odberu súčiastok počas 30 týždňov prevádzky. Cieľom simulácie bolo **nasimulovať celkové náklady** za skladovanie súčiastok ako aj náklady vzniknuté **pokutou** za nedodané súčiastky odberateľovi. Táto trieda zoskupuje logiku pre všetky navrhnuté stratégie, pričom definuje všetky potrebné premenné, ktoré sa využívajú počas behu simulácie ako napr. cena pokuty za nedodané súčiastky, cena za skladovanie jednotlivých súčiastok, využívané generátory, atď...

4.3.1 Použité generátory

Táto trieda obsahuje pri inicializácii vytvorené generátory, ktoré sa využívajú počas behu simulácie v závislosti od zvolenej stratégie.

Generátory pre náhodnú veľkosť objednávky odberateľa:

- **shockAbsorbersGenerator** – Generátor diskrétného rovnomerného rozdelenia $\langle 50; 100 \rangle$.
- **brakePlatesGenerator** – Generátor diskrétného rovnomerného rozdelenia $\langle 60; 250 \rangle$.
- **headlightsGenerator** – Generátor empirického rozdelenia so 4 intervalmi.

Generátory pre dodacie pravdepodobnosti dodávateľov:

- **supplier1First10Weeks** – Generátor spojitého rovnomerného rozdelenia $\langle 10; 70 \rangle$.
- **supplier1After10Weeks** – Generátor spojitého rovnomerného rozdelenia $\langle 30; 95 \rangle$.
- **supplier2First15Weeks** – Generátor spojitého empirického rozdelenia s 5 intervalmi.
- **supplier2After15Weeks** – Generátor spojitého empirického rozdelenia s 5 intervalmi.

Pomocné generátory:

- **probabilityGenerator** – Generátor pre náhodnú pravdepodobnosť $\langle 0; 100 \rangle$.

4.3.2 Inicializácia

Konštruktor inicializuje simuláciu a nastavuje všetky potrebné generátory a dáta. Prijíma inštanciu generátora seedov a názov **CSV súboru** pre načítanie konfigurácie pre požadovanú stratégiu. Taktiež vytvára intervaly potrebné pre inicializáciu vyššie

spomenutých empirických generátorov a spúšťa metódu **loadWeeklyOrders()** pre načítanie konfigurácie. Súčasťou zadania semestrálnej práce bolo aj vykresľovanie grafu zobrazujúci priebeh simulácie, aby som oddelil používateľské rozhranie od jadra aplikácie, do simulácie posielam **lambda funkciu**, pomocou ktorej aktualizujem graf, tá je reprezentovaná premennou **graphUpdater**.

4.3.3 Načítavanie konfigurácie z CSV súborov

Ako bolo vyššie spomenuté, táto trieda zoskupuje logiku pre rôzne navrhnuté stratégie, ktoré potrebujú rozličnú konfiguráciu. Tieto jednotlivé konfigurácie sú uložené v **CSV** súboroch, ktoré obsahujú informácie o objednávkach pre jednotlivé týždne, ktoré sú simulované. Všetky tieto načítané dáta sú ukladané do poľa **weeklyOrders**.

4.3.4 Priebeh simulácie – **beforeAllReplications()**

V tejto metóde sa mi pred vykonaním všetkých replikácií **resetuje** premenná pre **celkové náklady** a taktiež sa mi vypočítava **interval**, po koľkých replikáciách sa má na grafe vykresliť hodnota. Taktiež sa mi tu vypočítava **index replikácie**, po koľkú sa bude ignorovať vykresľovanie hodnôt v grafe.

Interval aktualizovania (každá 1000-ca replikácia, v prípade počtu replikácií menšieho ako 1000 je interval = 1) ako aj **ignorovaný index** (vždy presahuje prvých 15% replikácií) replikácie sa mi vypočítavajú z celkového počtu zadaných replikácií.

4.3.5 Priebeh simulácie – beforeReplication()

V tejto metóde sa pred začatím každej replikácie vynuluje sklad (premenné predstavujú počet daných súčiastok na sklade).

4.3.6 Priebeh simulácie – virtualReplication()

Táto metóda obsahuje hlavnú logiku simulácie. Predstavuje **30-týždňový** priebeh dodávania a odoberania zásob zo skladu, pričom **skladovaním súčiastok a platením pokút** za nedodanie súčiastok postupne vznikajú náklady pre podnikateľa Jána.

Obsahuje 2 cykly for – jeden pre týždne a druhý pre jednotlivé dni v týždni.

Každý týždeň sa vygeneruje pravdepodobnosť (probability) a pravdepodobnosť dodania od dodávateľa pomocou vyššie popísaných generátorov. Na základe načítanej konfigurácie sa vyberie dodávateľ pre daný týždeň a následne sa určí pomocou podmienok, ktorý generátor sa použije pre pravdepodobnosť dodania, keďže obaja dodávatelia majú rôznu spoľahlivosť na základe toho aký týždeň sa aktuálne simuluje. Následne sa tieto **pravdepodobnosti porovnajú** a určí sa, či bude dodávka úspešná alebo neúspešná.

Každý simulovaný deň sa kontroluje či sa nejedná o pondelok alebo piatok:

- Pondelok – pokiaľ je dodávka úspešná, pričíta sa množstvo objednaných súčiastok (z konfigurácie) do skladu.
- Piatok – odberateľ odoberie náhodne vygenerovaný počet súčiastok a v prípade, že nie je dostatočné množstvo na sklade, započíta sa za toto chýbajúce množstvo pokuta do celkových nákladov.

Na konci každého dňa sa pripočítajú náklady za skladovanie podľa aktuálneho množstva jednotlivých súčiastok na sklade.

V prípade, že používateľ vyžaduje **zobrazenie iba jednej replikácie**, bolo zadané v zadání semestrálnej práce, že sa má vykresliť graf zobrazujúci postupný **vývoj celkových nákladov pre jednotlivé dni**. V takomto prípade sa na konci každého dňa ešte vykreslí aktuálny stav celkových nákladov pre daný deň pomocou vyššie spomenutej **lambda funkcie graphUpdater**.

4.3.7 Priebeh simulácie – afterReplication()

V tejto metóde sa po každej replikácii skontroluje, či sa má vykresľovať na základe zadania graf, zobrazujúci priemerné celkové náklady pre všetky replikácie (počet replikácií väčší ako 1).

Následne sa kontroluje či bola zadefinovaná lambda funkcia pre vykresľovanie grafu a či po danej replikácii by mala byť vykreslená hodnota v grafe na základe intervalu aktualizácie ako aj indexu po ktorý sa ignoruje vykresľovanie. Pokiaľ sú tieto podmienky splnené, **hodnota sa vykreslí do grafu**.

4.4 Trieda BusinessmanJohnApplication

Trieda **BusinessmanJohnApplication** riadi simulácie pre podnikateľa Jána a umožňuje spúšťať rôzne stratégie objednávanie súčiastok. Táto trieda slúži na komunikáciu s používateľským rozhraním **GUI**.

4.4.1 Inicializácia

Konštruktor prijme seed na základe ktorého vytvorí **generátor seedov**, ktoré budú využívať všetky inicializované simulácie. Následne sa inicializujú tieto konkrétne stratégie (stratégie A-D a vlastná stratégia) pomocou pripravených konfiguračných **CSV súborov**.

4.4.2 Nastavenie vykresľovanie grafov

Pomocou metódy **setGraphUpdater()** je možné nastaviť pre simulácie **lambda funkciu na vykresľovanie grafov**. Táto funkcia, ktorú táto metóda prijme sa následne nastaví pre všetky vytvorené stratégie.

4.4.3 Spustenie simulácií

Pomocou metódy **runSimulation()** je možné spustenie **zvolenej stratégie** so **zadaným počtom replikácií**, ktoré táto metóda prijme v parametroch.

4.4.4 Zastavenie simulácie

Pomocou metódy **stopSimulation()** je možné zastavenie aktuálne bežiacej simulácie.

5 Popis implementácie GUI

V tejto časti stručne popisujem jednotlivé komponenty a funkcionality používateľského rozhrania pre túto semestrálnu prácu.

5.1 Trieda AppGUI

Trieda **AppGUI** poskytuje grafické používateľské rozhranie pre ovládanie aplikácie a spúšťanie simulácií pre podnikateľa Jána. Umožňuje výber stratégie, nastavenie počtu replikácií zvolenej simulácie a vizualizáciu výsledkov v grafe.

Používateľské rozhranie obsahuje tieto komponenty:

- **Hlavné okno** – obsahuje rozhranie s možnosťami pre nastavenie simulácií.
- **Checkboxy pre výber stratégie** – umožňuje vybrať jednu z piatich vyššie spomenutých stratégií.
- **Input okno** – pre zadanie počtu replikácií simulácie.
- **Tlačidlá** – pre spustenie (Run Simulation) a zastavenie simulácie (Stop Simulation).
- **Graf** – graf vytvorení pomocou knižnice **JFreeChart** zobrazujúci vývoj celkových nákladov na základe zvoleného počtu replikácií.

6 Záver

V rámci tejto semestrálnej práce som implementoval Monte Carlo simuláciu pre optimalizáciu objednávok pre podnikateľa Jána, ktorý spravuje sklad so súčiastkami. Simulácia modeluje dodávky od dvoch dodávateľov, spotrebu súčiastok odberateľom, skladovacie náklady a pokuty za nedodanie.

System podporuje rôzne stratégie objednávaní, ktoré sú testované v simulácii s možnosťou vizualizácie výsledkov do grafu.

Grafické rozhranie umožňuje jednoduché ovládanie, výber stratégie a zobrazenie výsledkov v reálnom čase.

Výsledky experimentov umožňujú porovnanie stratégií a hľadanie optimálnej voľby, ktorá minimalizuje celkové náklady spojené so skladovaním a pokutami za nedodanie.